

РАЗГОНЯЕМ API: КАК СРЕЗАТЬ ВРЕМЯ ОТКЛИКА С 7000 ДО 6 МС

Backend developer, Центр исследований и разработки Сбера в Нижнем Новгороде

Опыт разработки — 2 года

Родные языки — C#, Python



Эффект “Черная пятница”

Резкий, лавинообразный и часто экстремальный всплеск пользовательского трафика и транзакций, с которым сталкиваются веб-сайты, приложения и платежные шлюзы во время крупных распродаж.



Почему это опасно

Симптомы:

1. Резкий скачок Latency
2. Полка по CPU и памяти
3. Переполнение пула соединений с БД
4. Всплеск ошибок 5xx
5. «Троттлинг» сторонних API

Последствия

Получаем:

1. Отток клиентов
2. Потеря прибыли
3. Репутационный ущерб

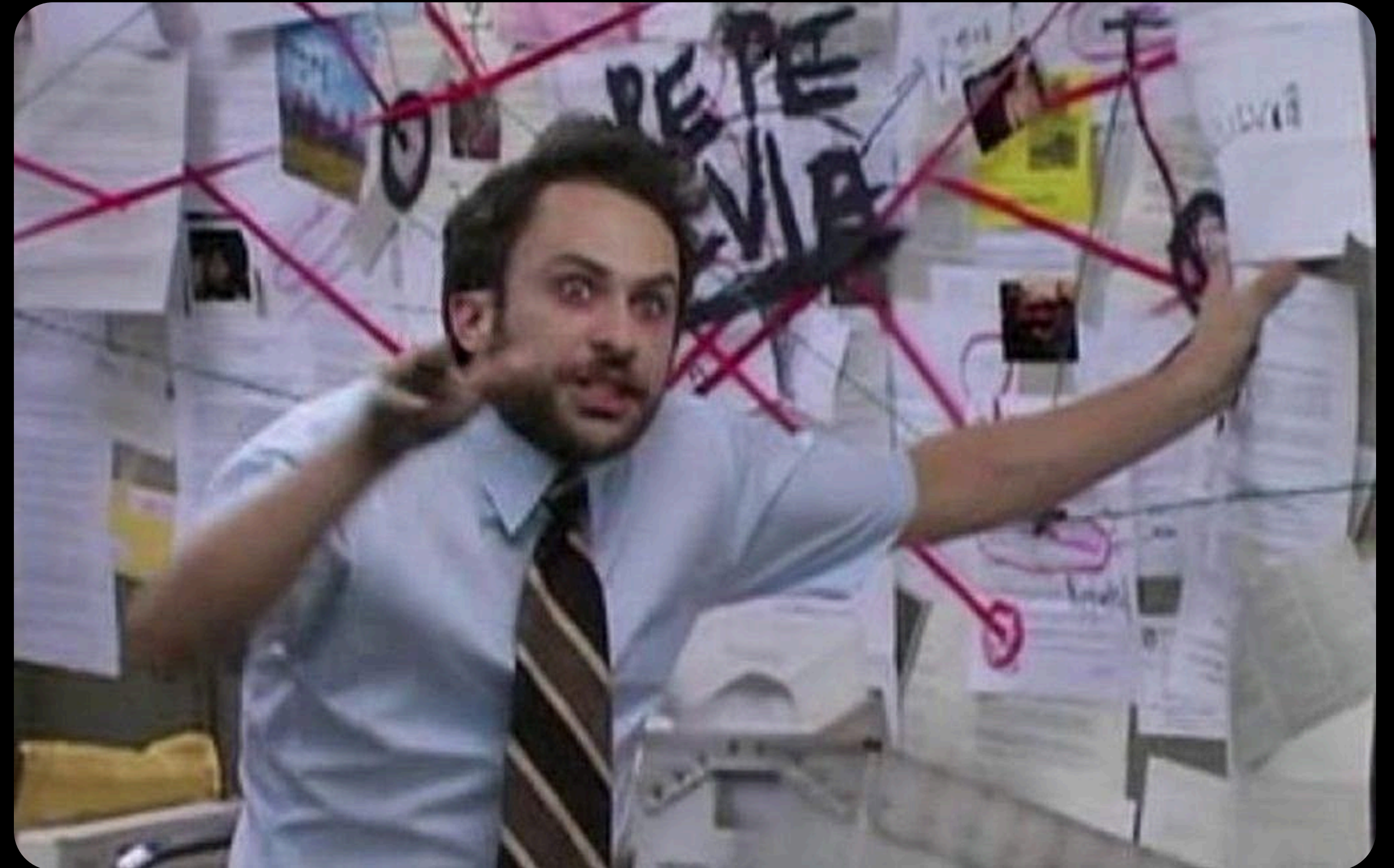


Немного теории

Проблемы:

1. I/O задержки.
2. Сетевые задержки.
3. Интерпретация SQL.
4. Сериализация данных.

Решение есть! Кэширование.

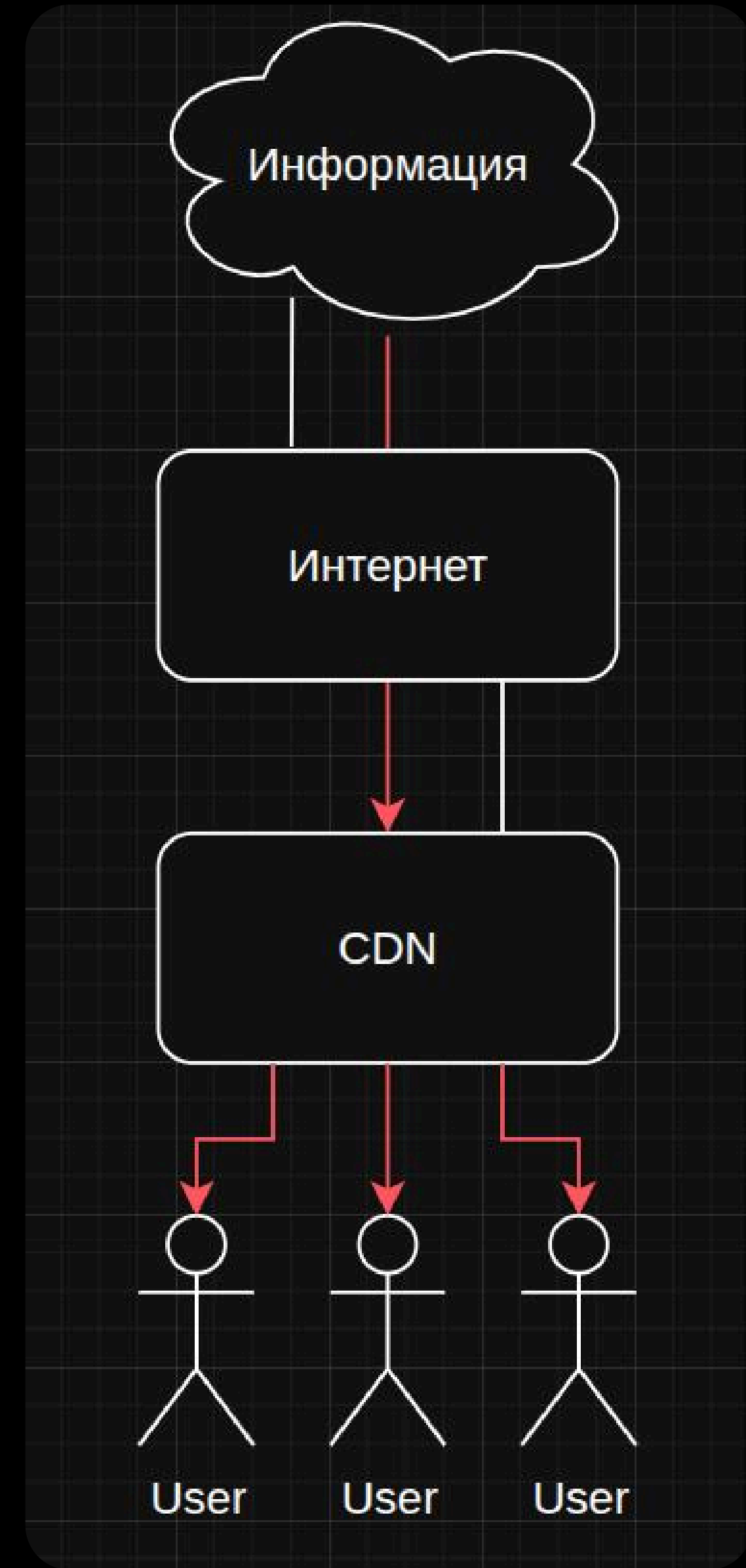


Топология кэширования

Где мы можем перехватывать запросы?

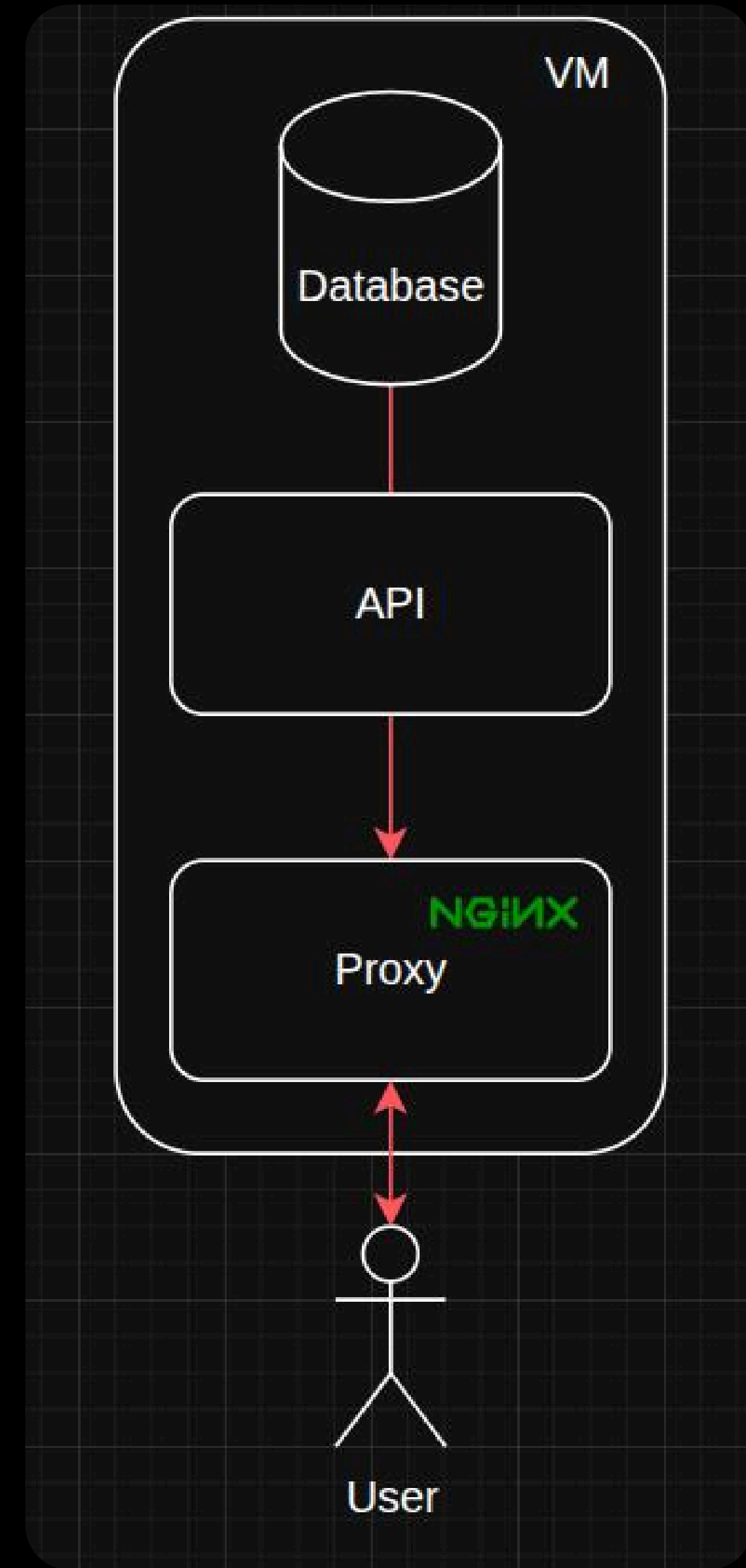
Edge / CDN

Запрос перехватывается
“соседними” дата-центрами и
возвращает данные за
считанные миллисекунды.



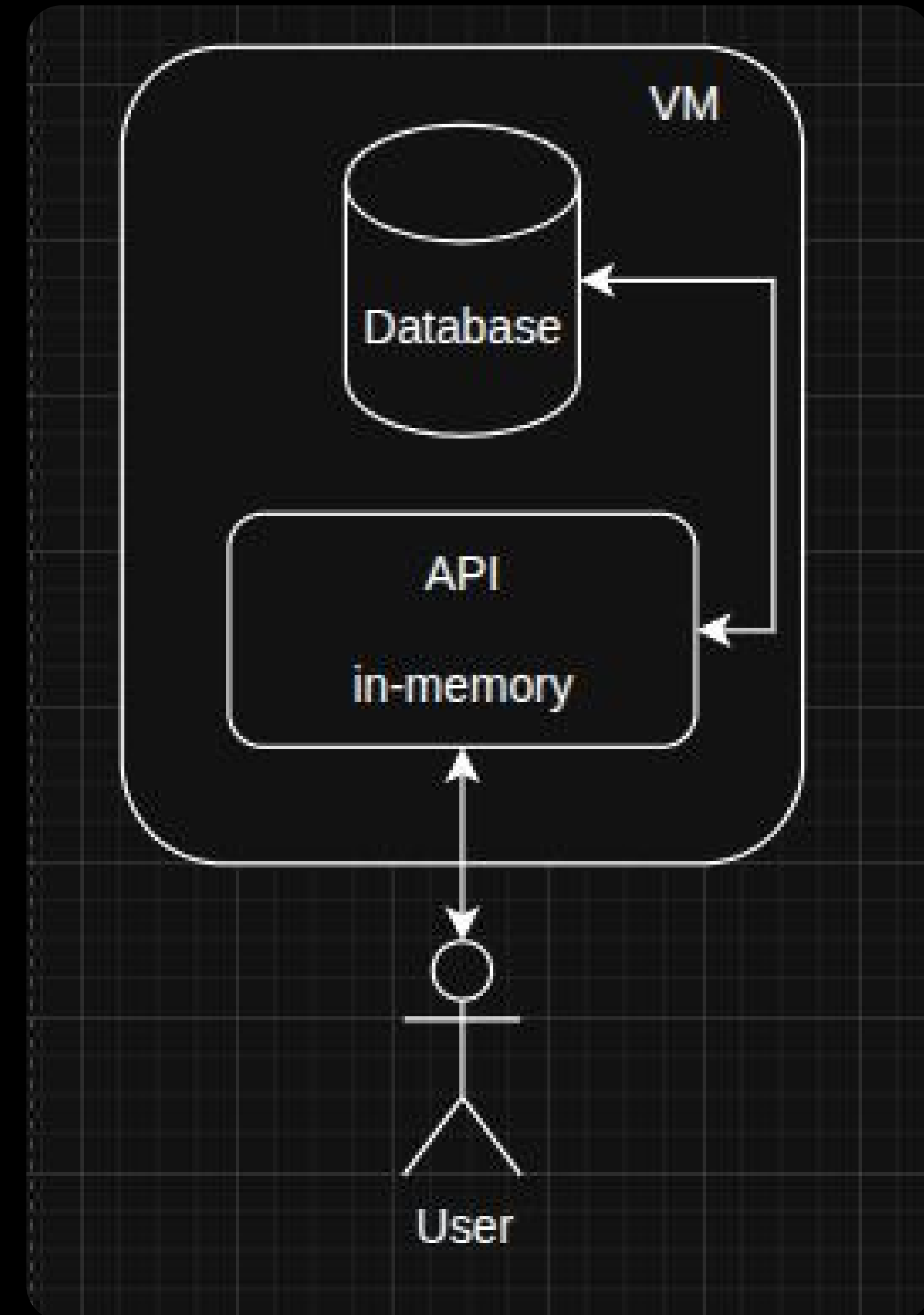
API Gateway / Reverse Proxy (Nginx)

Запрос не доходит до API/
сервиса, слой proxy сам отдает
закешированные данные.



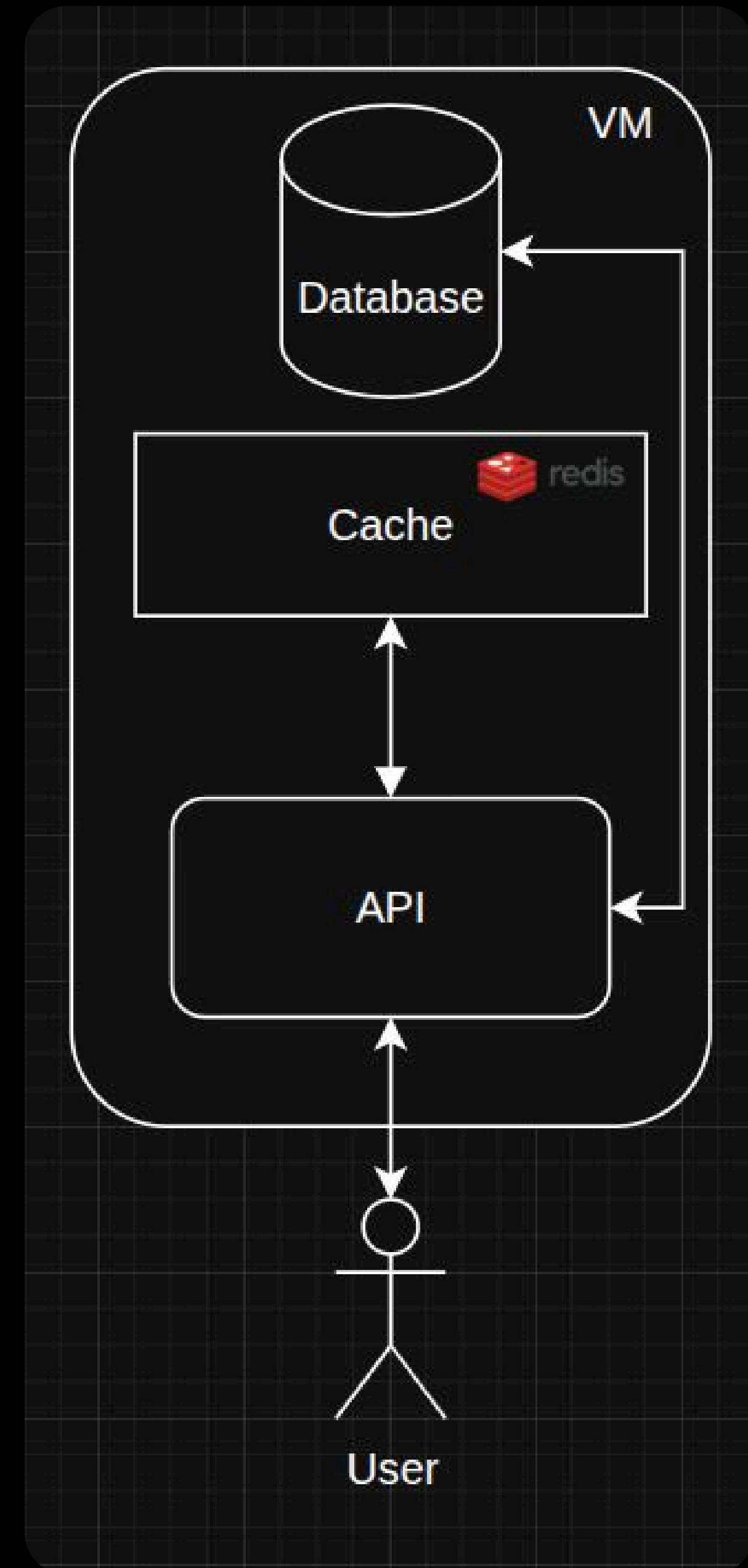
Уровень приложения (In-Memory, L1):

Хранится непосредственно в оперативной памяти нашего процесса (API).



Уровень распределенного
кэша (Redis / Memcached, L2):

Данные хранятся в
оперативной памяти другого
процесса.

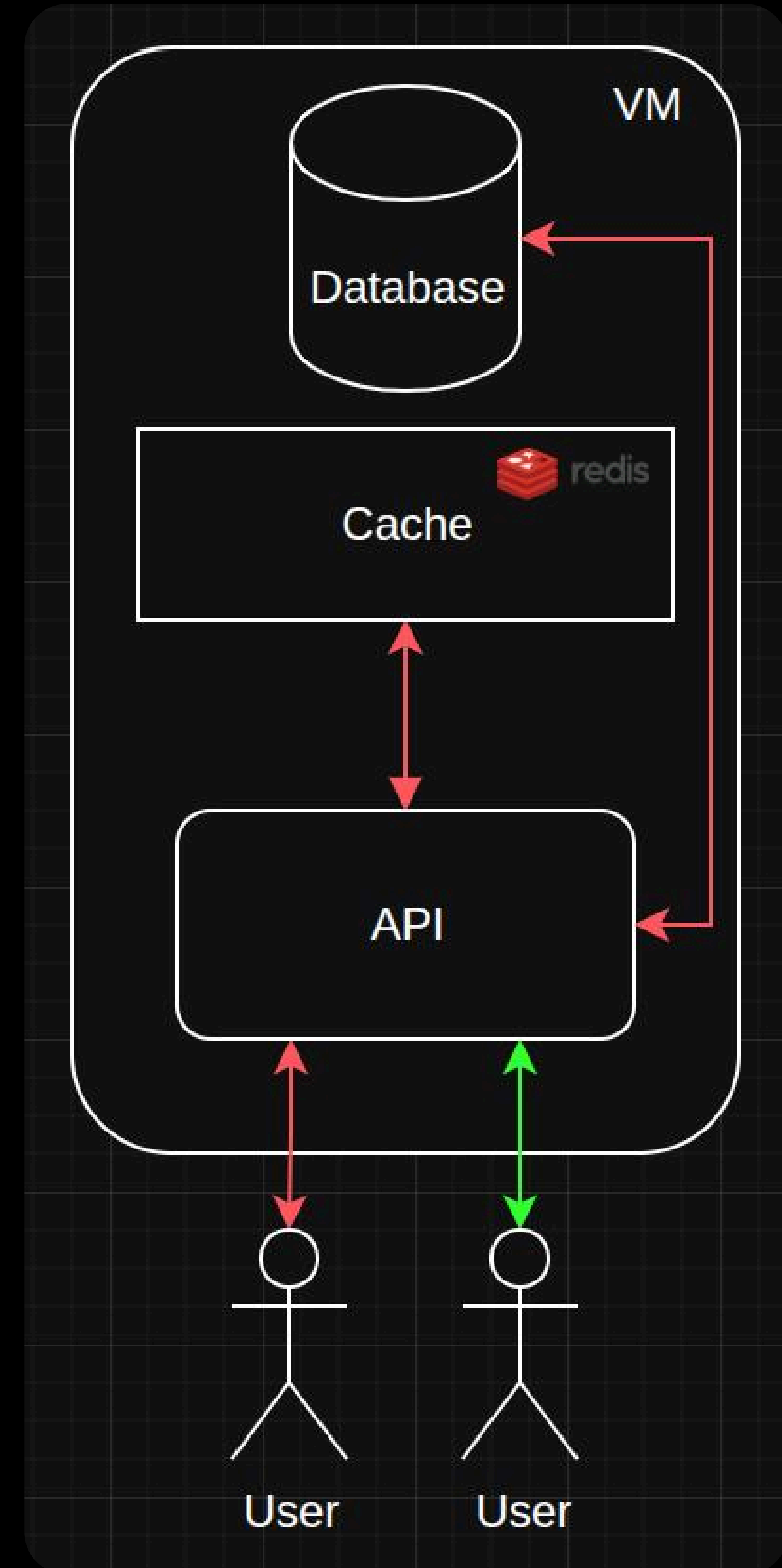


Стратегия работы с КЭШем

Cache-Aside (Ленивая загрузка)

Данные не попадают в кэш заранее.

Наш выбор!



Важные термины

TTL (Time-to-live)	L1 = 30 сек. - 2 мин. L2 = 2 мин. - 5 мин.
Вытеснение данных (Eviction Policies)	LRU - дольше всего не используемые LFU - самые редко используемые данные

Итак...

Практика

Cache-Aside + L1/L2 + Request Coalescing

Контекст

- 500 синхронных процессов
- `get endpoint`
- `payload: 10` пар `key-value`
- синтетическая нагрузка

Сценарий без кэша

Запросов/сек:
73/s

Latency p(95):
7.25s

```
semao0@semao0-AB350M-DS3H-V2:~/Projects/CacheExample/cache-example/LoadTest$ k6 run -e MODE=nocache -e VUS=500 test.js

Grafana

execution: local
script: test.js
output: -

scenarios: (100.00%) 1 scenario, 500 max VUs, 1m20s max duration (incl. graceful stop):
  over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
  x 'p(95)<1000' p(95)=7.25s

TOTAL RESULTS

checks_total.....: 3738    73.632473/s
checks_succeeded...: 100.00% 3738 out of 3738
checks_failed.....: 0.00%   0 out of 3738

HTTP
http_req_duration.....: avg=5.57s min=706.53ms med=5.95s max=8.25s p(90)=7.11s p(95)=7.25s
  { expected_response:true }...: avg=5.57s min=706.53ms med=5.95s max=8.25s p(90)=7.11s p(95)=7.25s
http_req_failed.....: 0.00%   0 out of 3738
http_reqs.....: 3738    73.632473/s

EXECUTION
iteration_duration.....: avg=5.67s min=807.8ms med=6.05s max=8.35s p(90)=7.21s p(95)=7.35s
iterations.....: 3738    73.632473/s
vus.....: 114    min=48    max=500
vus_max.....: 500    min=500    max=500

NETWORK
data_received.....: 1.2 MB 23 kB/s
data_sent.....: 344 kB 6.8 kB/s

running (0m50.8s), 000/500 VUs, 3738 complete and 0 interrupted iterations
default ✓ [=====] 000/500 VUs 50s
ERRO[0051] thresholds on metrics 'http_req_duration' have been crossed
```

Сценарий с L2 кэшем (Redis)

Запросов/сек:
3690/s!

Latency p(95):
16.84ms

```
• semao0@semao0-AB350M-DS3H-V2:~/Projects/CacheExample/cache-example/LoadTest$ k6 run -e MODE=redis -e VUS=500 test.js

Grafana

execution: local
script: test.js
output: -

scenarios: (100.00%) 1 scenario, 500 max VUs, 1m20s max duration (incl. graceful stop):
* default: Up to 500 looping VUs for 50s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<1000' p(95)=16.84ms

TOTAL RESULTS

checks_total.....: 184717 3689.414199/s
checks_succeeded...: 100.00% 184717 out of 184717
checks_failed.....: 0.00% 0 out of 184717

HTTP
http_req_duration.....: avg=7.6ms min=659.09µs med=4.28ms max=3.63s p(90)=12.57ms p(95)=16.84ms
{ expected_response:true }...: avg=7.6ms min=659.09µs med=4.28ms max=3.63s p(90)=12.57ms p(95)=16.84ms
http_req_failed.....: 0.00% 0 out of 184717
http_reqs.....: 184717 3689.414199/s

EXECUTION
iteration_duration.....: avg=108.41ms min=100.76ms med=105.04ms max=3.73s p(90)=113.76ms p(95)=117.97ms
iterations.....: 184717 3689.414199/s
vus.....: 5 min=5 max=500
vus_max.....: 500 min=500 max=500

NETWORK
data_received.....: 58 MB 1.2 MB/s
data_sent.....: 17 MB 332 kB/s

running (0m50.1s), 000/500 VUs, 184717 complete and 0 interrupted iterations
default ✓ [=====] 000/500 VUs 50s
```


Сценарий с L2 + L1
кэшем (+Request
Coalescing)

Запросов/сек:
3880/s

Latency p(95):
5.76ms!

```
• semao0@semao0-AB350M-DS3H-V2:~/Projects/CacheExample/cache-example/LoadTest$ k6 run -e MODE=hybrid -e VUS=500 test.js

Grafana

execution: local
script: test.js
output: -

scenarios: (100.00%) 1 scenario, 500 max VUs, 1m20s max duration (incl. graceful stop):
* default: Up to 500 looping VUs for 50s over 3 stages (gracefulRampDown: 30s, gracefulStop: 30s)

THRESHOLDS

http_req_duration
✓ 'p(95)<1000' p(95)=5.76ms

TOTAL RESULTS

checks_total.....: 194047 3877.307309/s
checks_succeeded...: 100.00% 194047 out of 194047
checks_failed.....: 0.00% 0 out of 194047

HTTP
http_req_duration.....: avg=2.56ms min=194.15µs med=1.32ms max=2.04s p(90)=4.21ms p(95)=5.76ms
{ expected_response:true }...: avg=2.56ms min=194.15µs med=1.32ms max=2.04s p(90)=4.21ms p(95)=5.76ms
http_req_failed.....: 0.00% 0 out of 194047
http_reqs.....: 194047 3877.307309/s

EXECUTION
iteration_duration.....: avg=103.2ms min=100.29ms med=101.96ms max=2.14s p(90)=105.07ms p(95)=106.66ms
iterations.....: 194047 3877.307309/s
vus.....: 6 min=6 max=500
vus_max.....: 500 min=500 max=500

NETWORK
data_received.....: 62 MB 1.2 MB/s
data_sent.....: 18 MB 353 kB/s

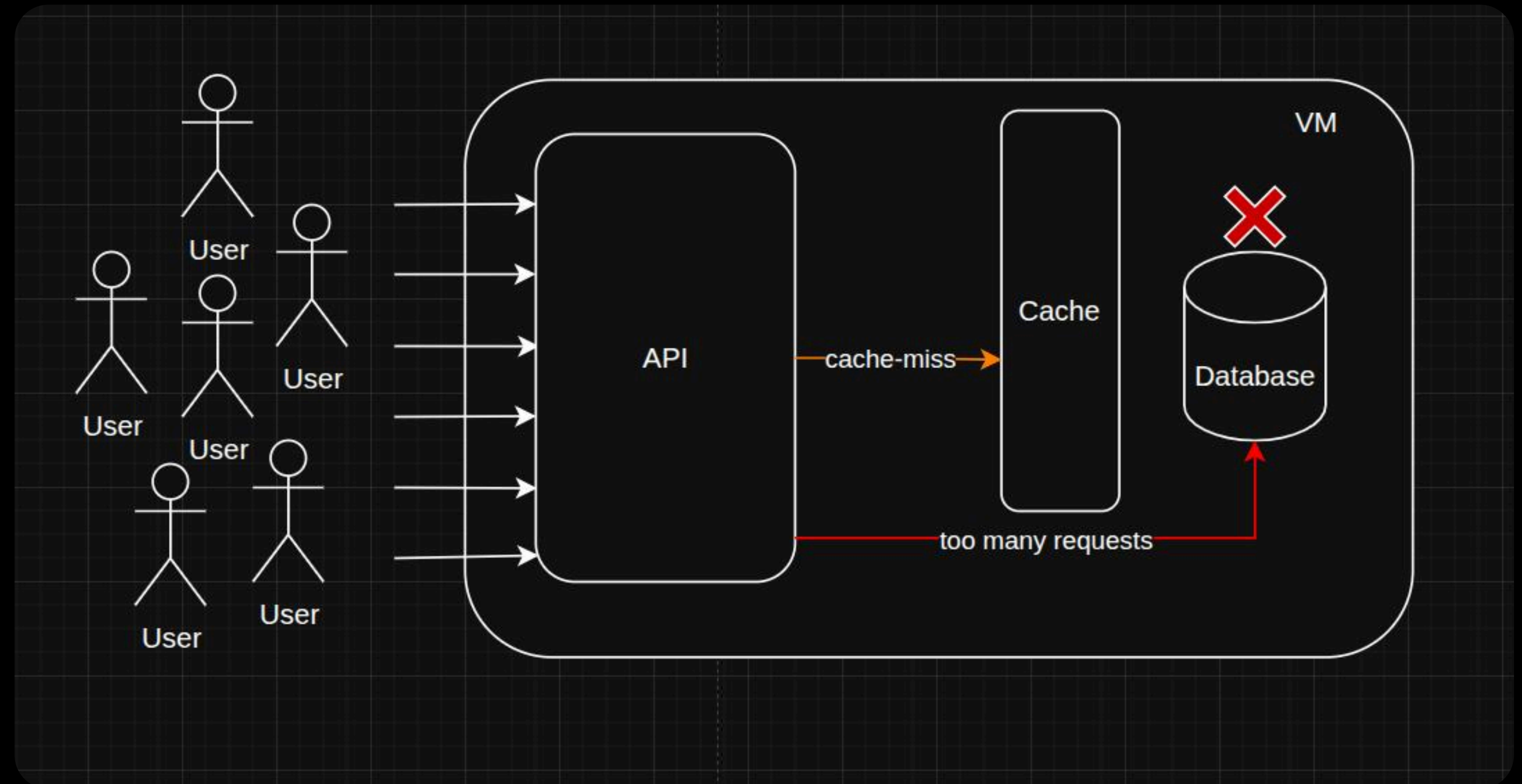
running (0m50.0s), 000/500 VUs, 194047 complete and 0 interrupted iterations
default ✓ [=====] 000/500 VUs 50s
```


НО...

Архитектурные вызовы и “грабли”

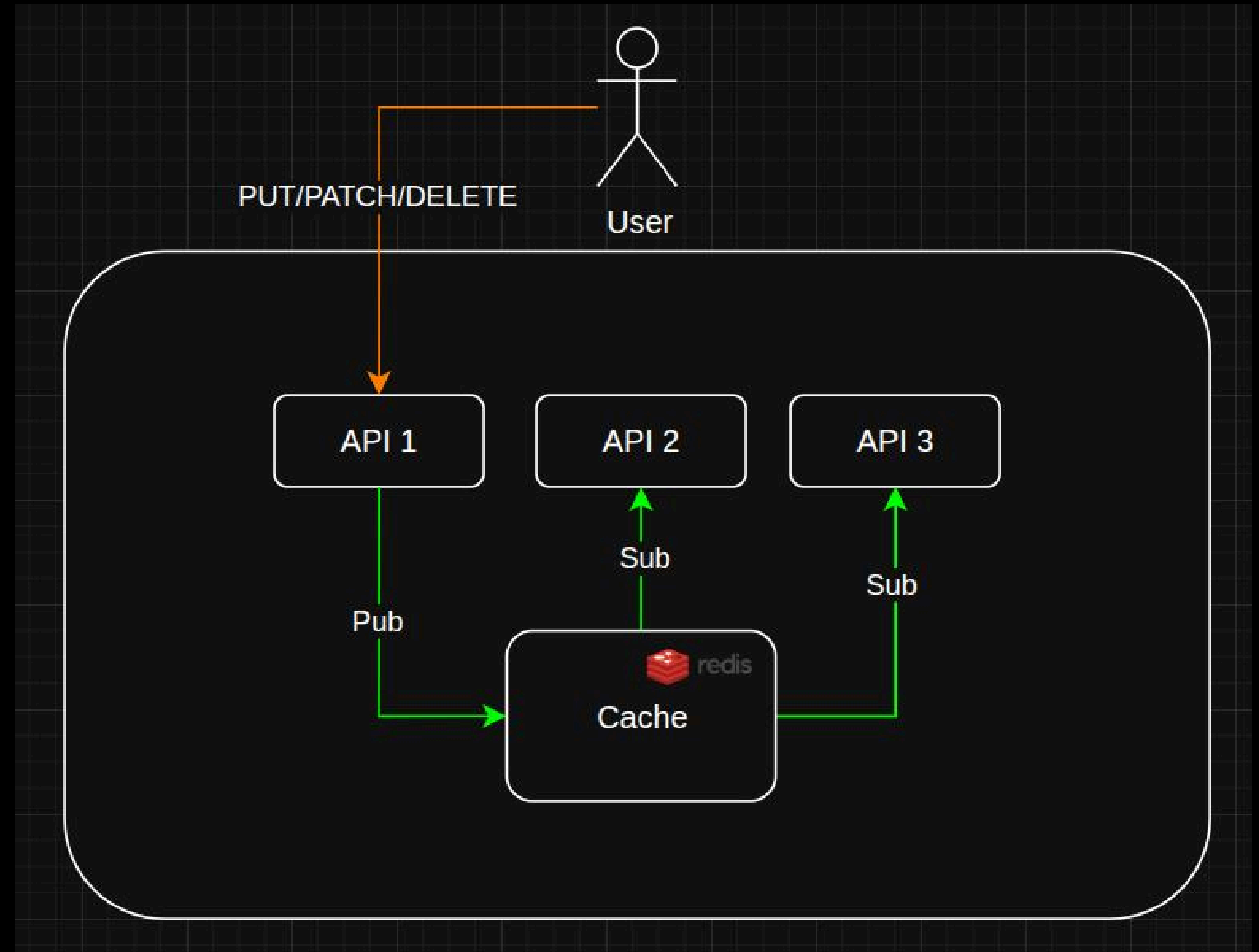
Cache Stampede (Эффект "Громового стада")

Разрешается
блокировкой
ПОТОКОВ



Рассинхронизация L1 (Stale Data)

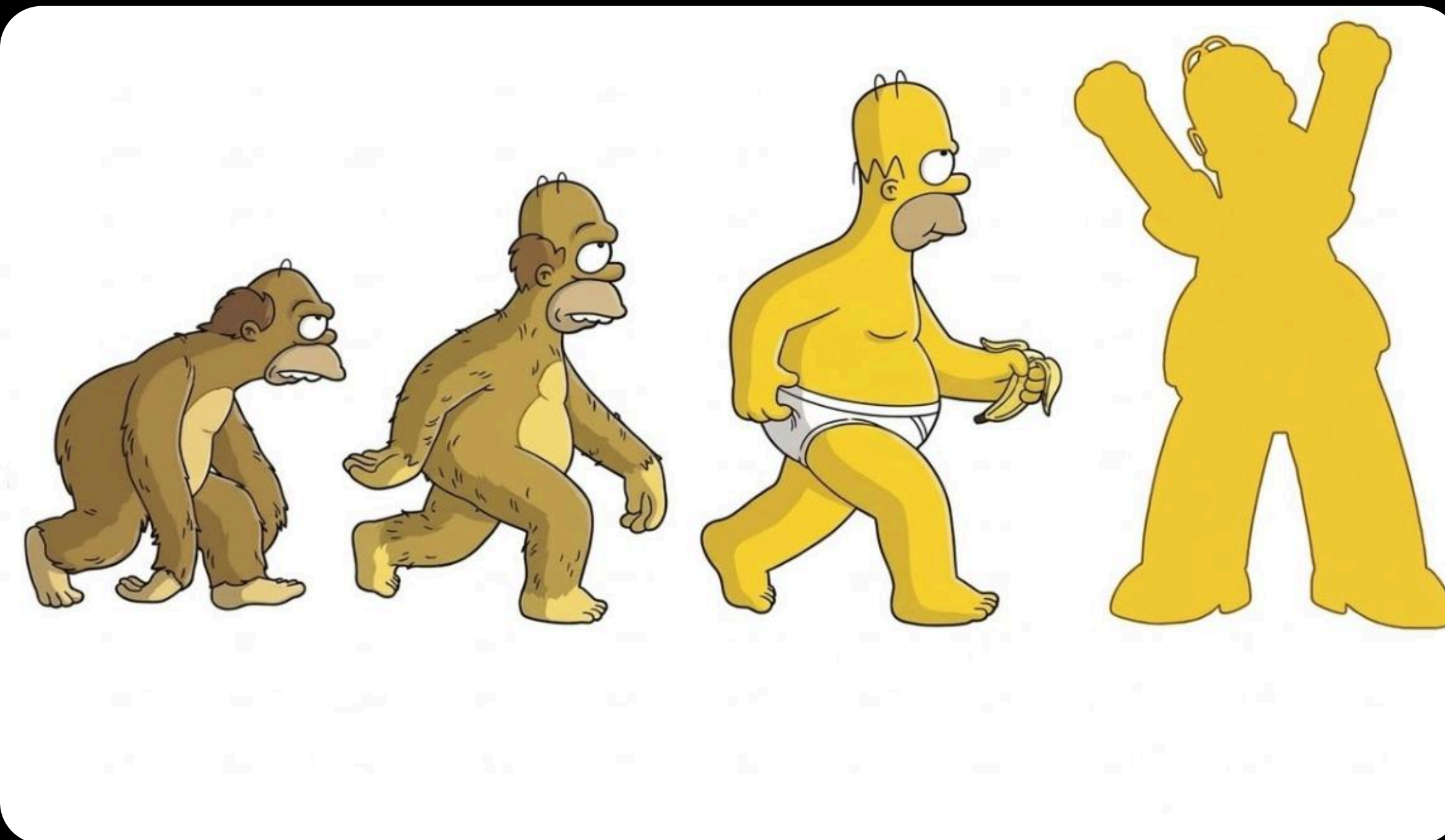
Решается
использованием
Redis Pub/Sub -
брокер сообщений



ВЫВОДЫ

Было: 73 RPS, 7+ сек. latency

Стало: 3880 RPS, 6 мс. latency



**«Дай мне удачу, чтобы без кэша можно
было обойтись, дай мне сил, если
обойтись без него нельзя – и дай мне
мудрости отличить одну ситуацию от
другой»**

Ждём на стажировку

R&D-исследователей:

- анализ рынка отраслевых AI-решений
- исследования потребностей целевой аудитории
- оценка потенциала монетизации решений

Продакт-менеджеров:

- управление бэклогом продукта и релизными циклами
- исследования и коммуникации с пользователями и стейкхолдерами

Оставь заявку
на стажировку
в Сбере!

